Python Cheat Sheet

MSc Digital Scholarship 2026

Colab essentials

A notebook is a list of cells: text cells and code cells. Run them in order, from the top.

```
Shift + Enter        # run this cell
Runtime → Restart session and run all
File → Save a copy in Drive
File → Upload notebook
Insert → Code cell
```

The last line of a code cell is shown automatically. Use print() to show anything else.

Values and types

Every value has a type, and each type has its own operations.

```
"Bennet"          # str: text
39                # int: whole number
4.2               # float: decimal number
['Mr.', 'Bennet']    # list
True, False       # bool
type(39)          # → int
```

Variables

= gives a value a name. == asks whether two values are equal.

```
size = len(sample)     # size is now 196
size == 196            # → True
name == "bennet"       # capitals matter
```

Three ways to call a tool

```
len(sample)            # function(thing)
sample.lower()         # thing.method()
statistics.mean(x)     # module.function(thing)
```

Arguments go in the brackets, separated by commas. Most tools return a value: keep it by naming it.

Strings

Text in quotes. The quotes must match; use the other kind inside.

```
'Mr. Bennet'
'"Is that you?" he asked.'
```

String methods (s is any string)

```
s.lower()  s.upper()   # change case
s.count("Bennet")      # how many times
s.replace("a", "b")    # swap a for b
s.split()              # list of words
s.split(".")           # cut at every .
s.strip(".,!?")        # trim the ends
s.startswith("B")      # True or False
s.endswith(".")
len(s)                 # number of characters
```

Methods return a new string: the original never changes.

Lists

An ordered collection. Positions (indexes) start at 0; negative positions count from the end.

Make a list

```
words = sample.split()
ending = ['The', 'whole', 'street',
          'knew', 'by', 'noon.']
```

Get items and slices

```
ending[0]        # 'The': first
ending[-1]       # 'noon.': last
ending[1:3]      # ['whole', 'street']
ending[:2]       # first two
ending[-3:]      # last three
```

[start:stop] includes start and stops before stop. Leave a number out for "from the beginning" or "to the end".

Useful with lists

```
len(words)           # how many items
"Bennet" in words    # True or False
sorted(words)        # a sorted copy
words.append("x")    # add to the end
```

List comprehensions

Make a new list from an old one. Read from the middle outwards.

```
# transform every item
[len(w) for w in words]
# keep only some items
[w for w in words if len(w) > 5]
# both at once
[w.lower() for w in words if w.isalpha()]
```

[what to make for each item in a list if a test (optional)]

for loops

Do something for each item. Colon at the end; indent the body by four spaces.

```
for w in words[0:3]:
    print(w, len(w))
```

To make a list: comprehension. To do something, such as printing: loop.

Conditions

Questions whose answer is True or False.

```
==  !=          # equal, not equal
<  >  <=  >=    # comparisons
in   not in     # inside?  absent?
```

if, elif and else

```
if len(w) > 10:
    print("long")
elif len(w) > 5:
    print("medium")
else:
    print("short")
```

Writing functions

def names a function; return says what it gives back.

```
def clean(w):
    return w.strip('.,;:!?"').lower()

clean('"Bennet,')     # → 'bennet'

def concordance(words, key, width=5):
    ...               # width defaults to 5
```

Modules, packages, libraries

A module is a file of code you import. A package is a collection of modules. "Library" is used for either.

```
import statistics
statistics.mean(x)
statistics.median(x)
statistics.pstdev(x)   # standard deviation

import re              # regular expressions
import pandas as pd    # tables (nickname pd)
from nltk.tokenize import sent_tokenize
```

Setting up in Colab

```
import nltk
nltk.download('punkt_tab')  # splitter data
!pip install something      # if missing
```

Reading error messages

Read the last line first. The arrow in the message shows which line failed.

```
NameError       # unknown name: a typo,
                # Print, or a missing import
AttributeError  # no such method: s.Upper()
IndexError      # past the end: words[100]
TypeError       # wrong type: "Page " + 5
SyntaxError     # not Python: a missing
                # quote, bracket or colon
```

The worst error has no message: code that runs and gives the wrong answer.

Glossary

argument a value given to a function or method, inside its brackets

return value what a function gives back

string a piece of text, in quotes

list an ordered collection of items, in square brackets

index a position in a list, counting from 0

slice a run of items from a list: [start:stop]

method a function that belongs to a type of value: s.lower()

comprehension a new list made from an old one

module a file of Python code that you import

Text analysis recipes

From our notebooks. Each one hides a decision you should be able to state.

Download a Project Gutenberg book

```
import urllib.request
url = ("https://www.gutenberg.org/cache/"
       "epub/1342/pg1342.txt")
page = urllib.request.urlopen(url)
raw = page.read().decode("utf-8")
```

Remember: this includes Gutenberg's licence and front matter.

Words, using a pattern

```
def words_in(text):
    return re.findall(
        r"[A-Za-z]+(?:['’-][A-Za-z]+)*", text)
```

Sentences, using NLTK

```
sentences = sent_tokenize(text)
lengths = [len(words_in(s)) for s in sentences]
statistics.mean(lengths)
```

Where does a word occur?

```
[i for i, w in enumerate(words)
   if w == "pride"]
```

enumerate gives each item together with its position.

Most common words (often in AI code)

```
from collections import Counter
Counter(words).most_common(10)
```

A table with pandas

```
pd.DataFrame({"sentence": sentences,
              "words": lengths})
```

Working with AI-written code

Use AI to write and explain code. You are responsible for every number you report.

1. Test it on something you can check by hand.
2. Ask what it assumed: what is a word, a sentence, the text?
3. Read its explanation against what the code actually does.
4. Ask twice, and ask another AI. Same answer?
5. Look at the extremes: the shortest, longest, first and last results.
6. Report your decisions in your methods.

Code that runs without an error is not the same as code that is right.

Your data and Colab

Colab is not covered by the University's Google Workspace agreement, and its terms may allow your input to be used to train AI models.

Only put in public-domain texts, or your own writing that you'd be happy to make public. No unpublished research, interview data, restricted archives or personal information.

Getting help

```
help(len)        # built-in documentation
s.               # then Tab: list of methods
```

Search for the last line of an error message, or paste it into an AI and check its explanation by fixing the code yourself.